

Sobreajuste (Overfitting): El Modelo Memorizador

Cuando un modelo de aprendizaje automático «**memoriza**» los datos de entrenamiento en lugar de aprender los patrones subyacentes, cae en el sobreajuste. Esto ocurre cuando el modelo es **demasiado complejo** para la cantidad de datos disponibles o se entrena excesivamente, capturando ruido y fluctuaciones aleatorias.



Sobreajuste: Síntomas y Causas

Rendimiento Engañoso: Excelente en datos de entrenamiento, pero **pobre en datos nuevos**.

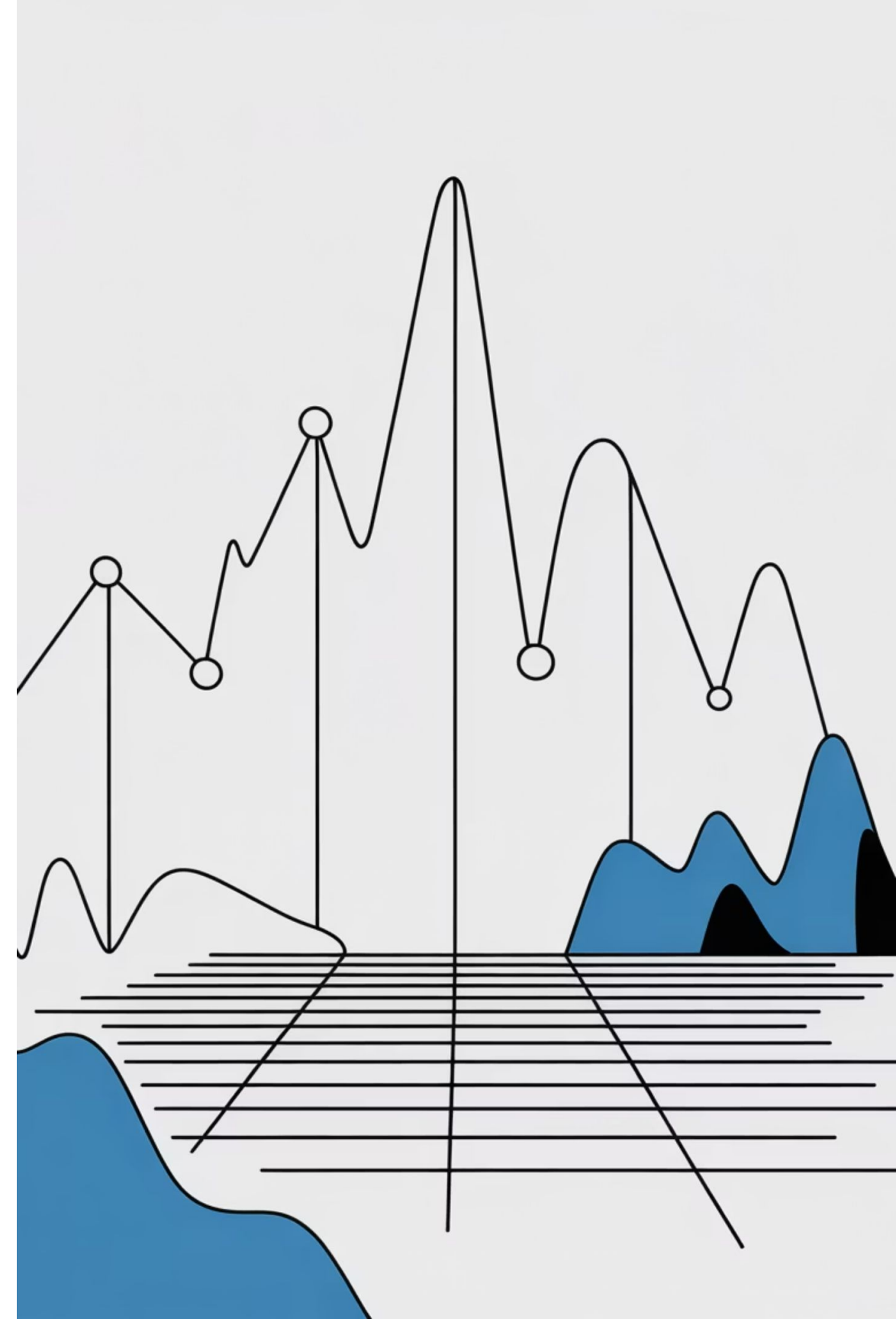
Captura de Ruido: Interpreta detalles insignificantes como reglas importantes.

Ejemplo Ilustrativo: Entrenar con outliers demasiado atípicos.



Subajuste (Underfitting): El Modelo Simplista

El subajuste ocurre cuando un modelo es **demasiado simple** para capturar la estructura inherente de los datos. No logra aprender los patrones subyacentes, lo que lleva a un **mal rendimiento** tanto en los datos de entrenamiento como en los nuevos.



Subajuste: Identificación y Origen

Falla en el Aprendizaje: El modelo no puede identificar las relaciones clave entre las variables.

Pobre Generalización: Su desempeño es deficiente incluso en los datos que ya ha visto.

Causas Frecuentes: Un modelo excesivamente simple, características insuficientes para la tarea, o un entrenamiento demasiado corto.

Caso Práctico: Un modelo lineal intentando predecir el comportamiento de la bolsa de valores, ignorando la complejidad de las interacciones económicas.



Modelo Demasiado Simple

No tiene la capacidad para aprender la complejidad de los datos.



Características Insuficientes

Faltan variables cruciales para el aprendizaje.



Entrenamiento Insuficiente

El modelo no ha tenido tiempo de ajustarse a los datos.

Equilibrio Óptimo: Sesgo (Bias) vs. Varianza

En el aprendizaje automático, encontrar el [equilibrio perfecto](#) entre sesgo y varianza es crucial para construir modelos robustos y precisos. Este dilema fundamental define la calidad de nuestras predicciones.

Sesgo (Bias)

Error por simplificaciones: Cuando el modelo hace suposiciones excesivamente sencillas sobre los datos.

Alto Sesgo: Conduce al **underfitting**. El modelo es inflexible y no captura la complejidad real de los datos.

“Asume demasiado” y **generaliza mal**, incluso sobre los datos con los que fue entrenado.

Modelo con ALTO SESGO (underfitting)

Usas una línea recta:

Precio = 1.0 × m²

Esto genera errores grandes:

m²	Precio real	Predicho
40	30	40 ✖
80	80	80 ✔
100	120	100 ✖

El modelo es demasiado simple para una relación que no es lineal.

Concepto	Significa
Sesgo (Bias)	Error por modelo demasiado simple
Underfitting	El modelo no aprende bien
Relación	Mucho sesgo → Underfitting

Varianza

Sensibilidad a los datos de entrenamiento: Mide cuánto cambia el modelo al entrenarse con diferentes conjuntos de datos.

Alta Varianza: Conduce al **overfitting**. El modelo es demasiado flexible y capta el ruido en lugar de los patrones generales.

Horas	Nota
1	50
2	55
3	58
4	65
5	70
6	72
7	78
8	80

Pero algunos estudiantes se enfermaron, copiaron, o tuvieron malos días:

Horas	Nota
3	30 ✖
6	95 ✖

Modelo con ALTA VARIANZA (overfitting)

Usas un polinomio grado 7 que pasa por todos los puntos:

- Predice perfecto los datos de training
- Aprende también los valores raros (30, 95)
- Para un estudiante nuevo que estudia 5 horas, puede predecir:

40 o 95, totalmente inestable

✖ Resultado:

- Error casi 0 en training
- Error muy alto en testing

Overfitting por alta varianza

Modelo con menor varianza

Usas una regresión simple o grado 2:

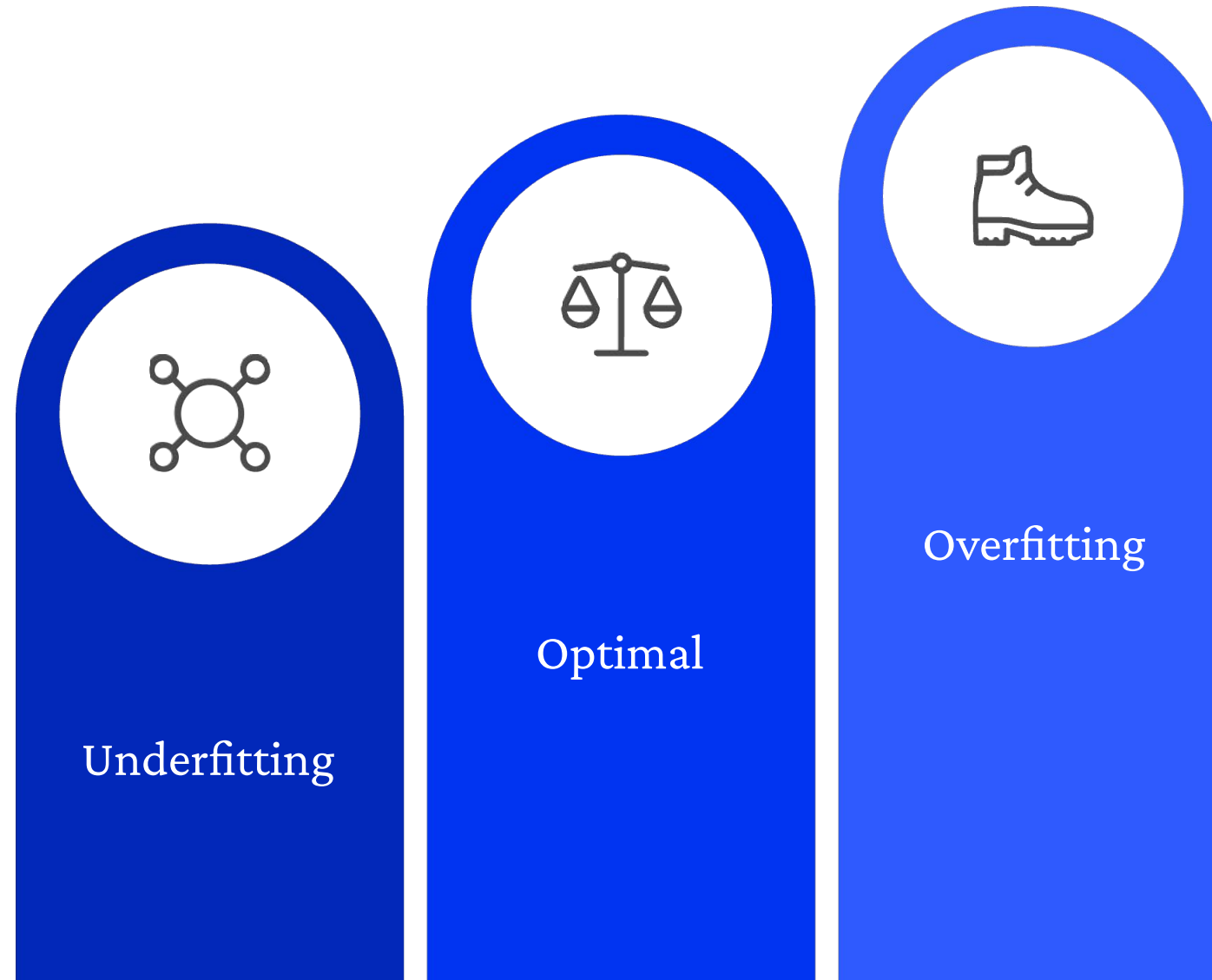
- No pasa por todos los puntos
- Ignora el ruido
- Captura la tendencia real:

más horas → mejor nota

Menor varianza

Mejor generalización

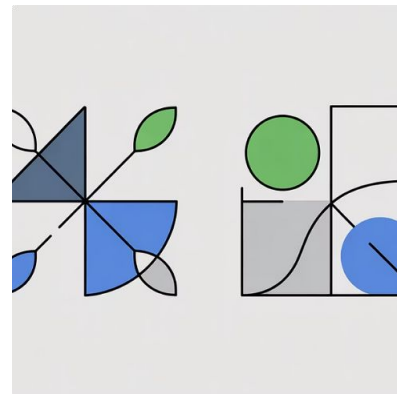
La Compensación Bias-Varianza



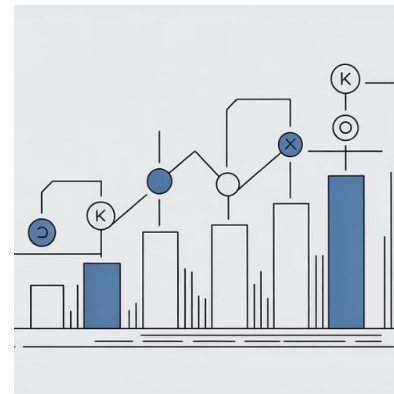
El objetivo es minimizar el error total, que es la suma del sesgo y la varianza. Al [reducir el sesgo](#), la varianza suele aumentar, y viceversa. La clave es encontrar el [punto óptimo](#) donde ambos se minimizan para un rendimiento generalizado superior.

Estrategias para el Equilibrio Bias-Varianza

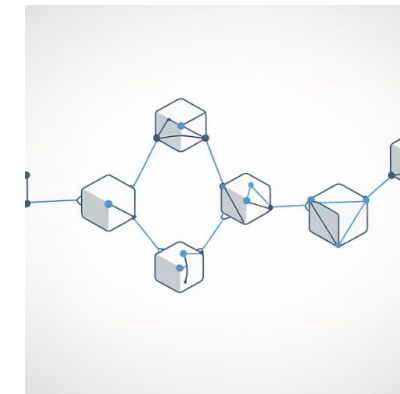
Lograr el equilibrio adecuado es un **arte y ciencia** en el Machine Learning. Aquí se presentan técnicas esenciales para afinar los modelos y minimizar el error total.



Regularización
Penaliza la complejidad del modelo, como L1 (Lasso) y L2 (Ridge), para reducir el sobreajuste y fomentar la generalización.



Validación Cruzada
Divide los datos en múltiples subconjuntos para entrenar y evaluar el modelo de forma robusta, proporcionando una estimación más fiable del rendimiento.



Conjuntos de Modelos (Ensembles)
Combina las predicciones de varios modelos (ej., Bagging, Boosting) para reducir el sesgo y la varianza, mejorando la precisión global.

L1 – Lasso: "Si no aportas, te vas"

Lasso castiga tanto que algunos coeficientes se vuelven **exactamente 0**.

Ejemplo:

Antes:

$$\text{Nota} = 2 \cdot \text{Horas} + 0.8 \cdot \text{Sueño} - 3 \cdot \text{Estrés} + 0.2 \cdot \text{Café}$$

Con Lasso:

$$\text{Nota} = 2 \cdot \text{Horas} + 0.8 \cdot \text{Sueño} - 3 \cdot \text{Estrés} + 0 \cdot \text{Café}$$

"Café" queda eliminado.

L2 – Ridge: "No te exageres"

Ridge castiga cuando los pesos son **muy grandes**.

No elimina variables

Las hace **más pequeñas**

Suaviza el modelo

Ejemplo:

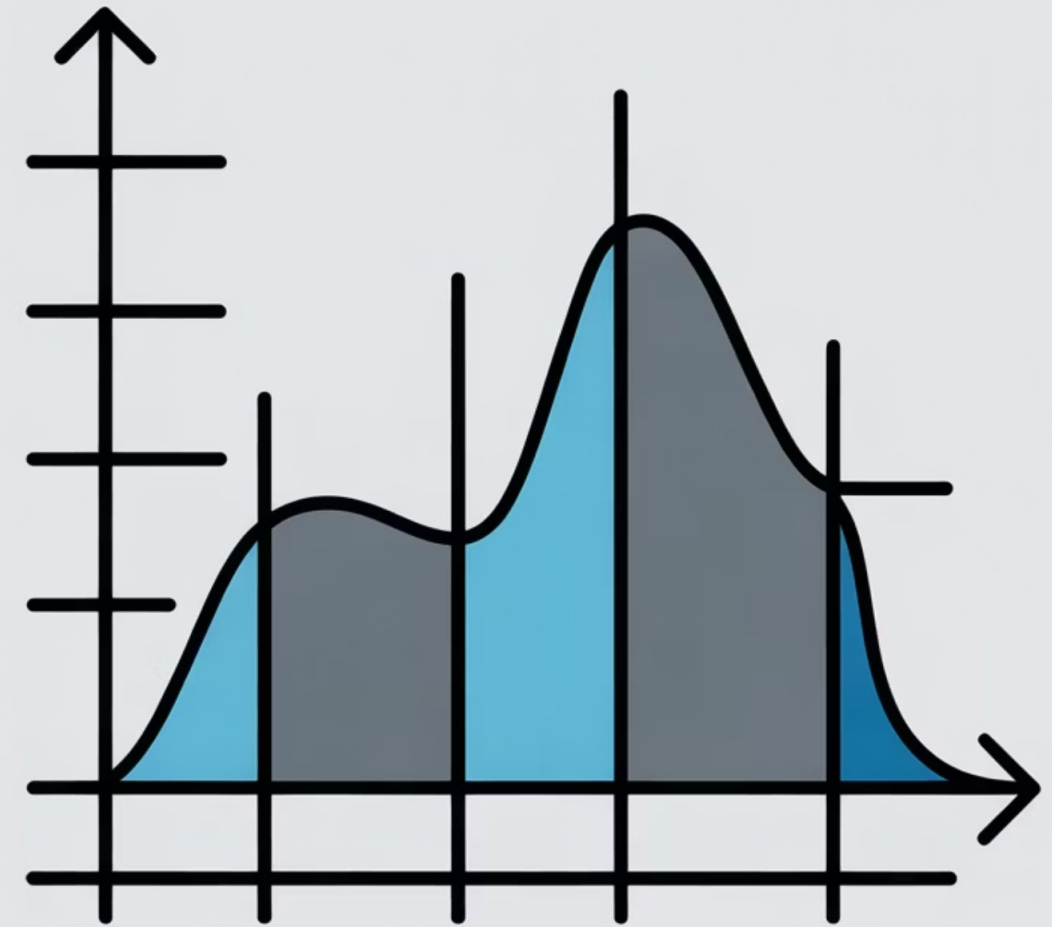
Antes: 12·Estrés

Ridge: 2·Estrés

Sigue usando la variable, pero con menos fuerza.

Regresión Logística: Clasificación Robusta y Eficiente

La regresión logística es un [modelo fundamental de clasificación](#) (a pesar de su nombre) que predice la probabilidad de un evento binario. Es un pilar en la industria por su [simplicidad, interpretabilidad y eficiencia computacional](#), sirviendo como base para entender algoritmos más complejos.



Fundamentos Matemáticos y Entrenamiento

La Función Sigmoide

Extiende la regresión lineal para clasificación utilizando la función logística:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Donde $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$. El resultado es una probabilidad entre 0 y 1, que se convierte en una decisión binaria mediante un umbral (típicamente 0.5).

Entrenamiento del Modelo

El objetivo es encontrar los coeficientes óptimos (β) que maximicen la verosimilitud (o minimicen la log-pérdida).

Aplicaciones Prácticas y Próximos Pasos



Diagnóstico Médico

Predecir la probabilidad de una enfermedad basándose en síntomas y datos clínicos.



Evaluación de Riesgo Crediticio

Determinar la solvencia de un cliente en el sector financiero.



Predicción de Conversión

Identificar clientes con alta probabilidad de realizar una compra online.

Para implementar estos conceptos, herramientas como [Scikit-learn \(Python\)](#) o [TensorFlow/PyTorch](#) son excelentes opciones. Explorar modelos como Árboles de Decisión, SVM o Redes Neuronales será el siguiente paso en vuestro aprendizaje.

Ejemplo

x_3^2 age	x_3^2 likes_pizza	x_3^2 attended_party
22	1	1
25	1	1
24	0	0
27	1	1
23	0	0
26	1	1
21	0	0
28	1	1
25	0	0
24	1	1